Multi-Threaded Web Server with IPC and Semaphores – Design Document

Diogo Martins 98501, Hélio Filho 93390

Table of Contents

1. Introduction
2. System Architecture
3. Component Design
4. Inter-Process Communication
5. Synchronization Analysis
6. Request Processing Flow
7. Data Structures

---

1. Introduction

1.1 Project Overview

This document describes the architecture and design of a multi-process, multi-threaded HTTP server implemented in C using POSIX APIs. The server follows a **Master/Worker** process model with per-worker thread pools for high concurrency.

1.2 Design Goals

Goal	Approach
High Concurrency	Multiple worker processes + thread pools
Safe Synchronization	POSIX semaphores, mutexes, rwlocks
Efficient I/O	`sendfile()` zero-copy, LRU file cache
Graceful Shutdown	Signal handling (SIGINT/SIGTERM)
Dynamic Content	CGI script execution via fork/exec
Configurability	File-based config + CLI overrides

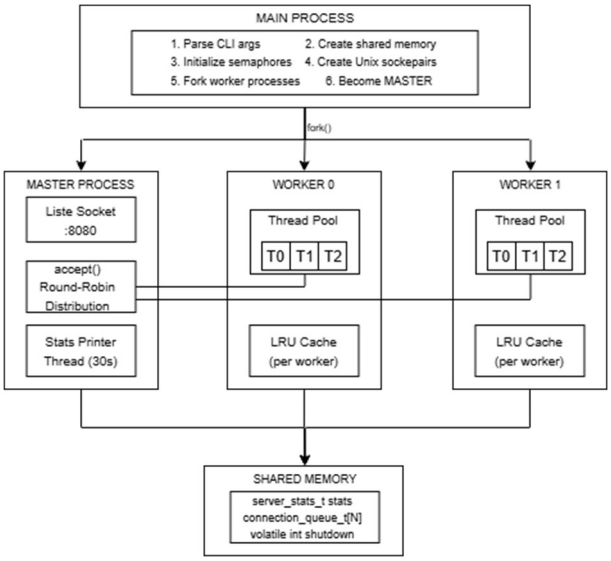
1.3 Source Files Overview

File	Purpose
`main.c`	Entry point, CLI parsing, process creation
`master.c/.h`	Master process: accept loop, FD distribution
`worker.c/.h`	Worker process: request handling coordination
`thread_pool.c/.h`	Per-worker thread pool (producer-consumer)
`http.c/.h`	HTTP protocol: responses, MIME types
`cgi.c/.h`	CGI script detection and execution
`lru_cache.c/.h`	LRU file cache with rwlock protection
`shared_mem.c/.h`	POSIX shared memory for statistics
`semaphores.c/.h`	Named semaphore initialization
`ipc_fd.c/.h`	File descriptor passing (SCM_RIGHTS)
`logger.c/.h`	Thread-safe access logging
`config.c/.h`	Configuration file parser
`stats.h`	Statistics data structures

---

2. System Architecture

2.1 High-Level Architecture Diagram



2.2 Process Modelhe server uses a pre-fork model:

1. **Main Process** initializes resources, then forks N worker processes
2. **Master Process** (parent after fork) handles the accept loop
3. **Worker Processes** (children) each run a thread pool for request processing

Benefits: - Process isolation: worker crash doesn't affect master - Per-worker resources: each worker has its own LRU cache - Scalability: can adjust worker count at startup

---

3. Component Design

3.1 Master Process (`master.c`)

Responsibilities: - Create and bind the listening socket (port 8080 default) - Accept incoming connections - Distribute connections to workers (round-robin with fallback) - Handle overload (503 when all queues full) - Run statistics printer thread (every 30 seconds)

Key Functions:

Function	Purpose
create_server_socket()	Create TCP socket with SO_REUSEADDR/SO_REUSEPORT
run_master()	Main accept loop
enqueue_connection()	Add connection to worker queue
stats_printer_thread()	Periodic statistics output

3.2 Worker Process (`worker.c`)

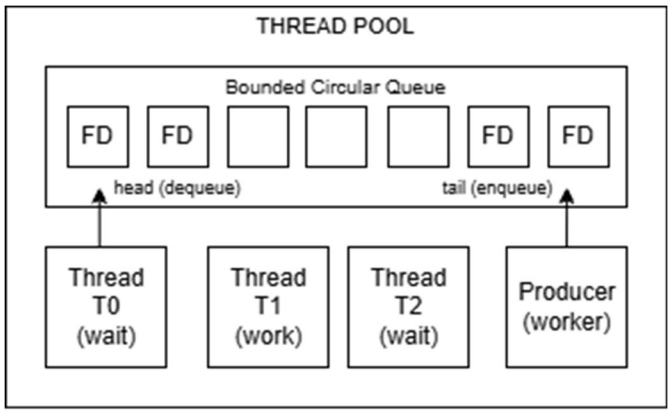
Responsibilities: - Receive file descriptors from master via Unix socket - Manage a thread pool for concurrent request handling - Maintain per-worker LRU cache - Process HTTP requests (static files, CGI, API endpoints)

Key Functions:

Function	Purpose
run_worker()	Main worker loop: dequeue tokens, receive FDs
process_client()	Handle HTTP request (called by pool threads)
dequeue_token()	Remove token from bounded queue
send_error_response()	Send error pages (404, 500, etc.)

3.3 Thread Pool (`thread_pool.c`)

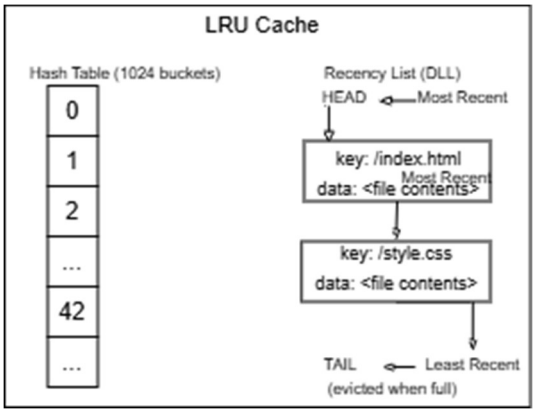
Design Pattern: Producer-Consumer with bounded queue



Synchronization: - `pthread_mutex_t mutex` — protects queue access - `pthread_cond_t cond` — signals queue state changes - Threads wait on condvar when queue empty - Producer signals when item added

3.4 LRU Cache (`lru_cache.c`)

Design: Hash table + Doubly-linked list

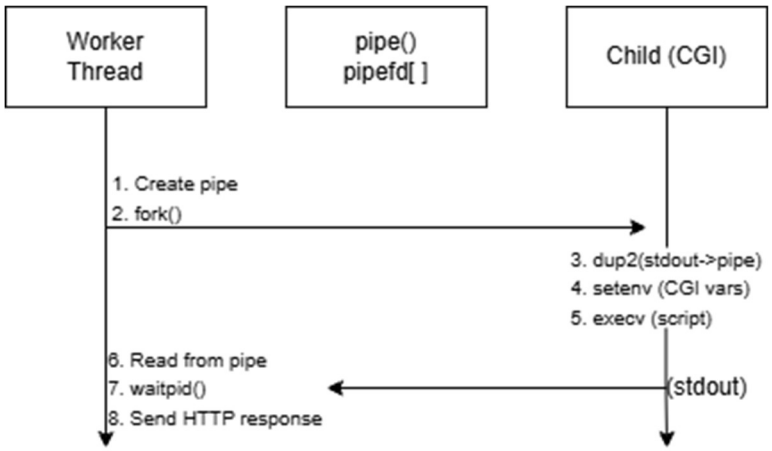


Configuration: - Max cache size: 10 MB per worker (configurable) - Max file size: 1 MB (files larger than this are not cached) - Eviction: LRU — tail entry removed when cache full

3.5 CGI Subsystem (`cgi.c`)

Detection: A request is CGI if: - Path contains `cgi-bin/`, OR - File ends with `.cgi` or `.sh`

Execution Flow:



Environment Variables Set: - `REQUEST_METHOD` (GET, HEAD) - `QUERY_STRING` (after `?` in URL) - `SCRIPT_NAME` (path to script) - `CONTENT_LENGTH` (for future POST support)

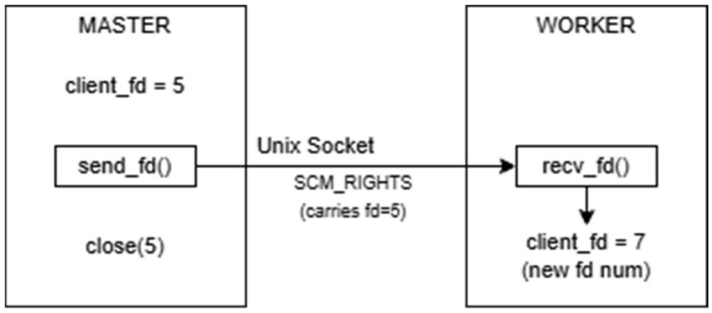
Limits: - Output capped at 1 MB (`CGI_MAX_OUTPUT`) - Script must be executable (`stat()` check)

---

4. Inter-Process Communication

4.1 Unix Socket FD Passing (`ipc_fd.c`)

File descriptors are passed from Master to Workers using Unix domain sockets with `SCM_RIGHTS` ancillary data.



Implementation: - `sendmsg()` with `CMSG_DATA` containing the FD - `recvmsg()` extracts FD from control message - Kernel duplicates FD into receiver's file table

4.2 Shared Memory (`shared_mem.c`)

POSIX shared memory (`shm_open()` + `mmap()`) provides cross-process data sharing.

Shared Data Structure:

```
typedef struct {
    connection_queue_t worker_queues[MAX_WORKERS];   // Per-worker queues
    server_stats_t   stats;                          // Global statistics
    volatile   int   shutdown;                       // Shutdown flag
}  shared_data_t;
```

Memory Layout:

```
/webserver_shm

worker_queues[0]
  ├─ tokens[256]  (circular buffer)
  ├─ front, rear, count

worker_queues[1]
  ├─ tokens[256]
  ├─ front, rear, count

...

stats
  ├─ total_requests (uint64_t)
  ├─ bytes_transferred (uint64_t)
  ├─ cache_hits, cache_misses (uint64_t)
  ├─ status_counts[600] (uint32_t[])
  ├─ active_connections (uint32_t)
  ├─ avg_response_time (double)
  └─ start_time (time_t)

shutdown (volatile int)
```

5. Synchronization Analysis

5.1 Synchronization Primitives Overview

Primitive	Type	Purpose	Scope
`empty_slots[i]`	Counting semaphore	Track free queue slots	Per-worker
`filled_slots[i]`	Counting semaphore	Track pending items	Per-worker
`queue_mutex[i]`	Binary semaphore	Protect queue access	Per-worker
`stats_mutex`	Binary semaphore	Protect statistics	Global
`log_mutex`	Binary semaphore	Protect log file	Global
`pool->mutex`	pthread_mutex_t	Protect thread pool queue	Per-pool
`pool->cond`	pthread_cond_t	Signal queue changes	Per-pool
`cache->lock`	pthread_rwlock_t	Protect LRU cache	Per-worker

5.2 Connection Queue Synchronization (Producer-Consumer)

Semaphore Setup (per worker):

```
empty_slots[i] = queue_size (256)   // Available slots
filled_slots[i] = 0                 // Pending items
queue_mutex[i] = 1                  // Binary mutex
```

Producer (Master) — Enqueue:

```
MASTER: enqueue_connection()

1. sem_trywait(empty_slots[worker])  // Non-blocking
   └─ If fails: try other workers or return 503

2. sem_wait(queue_mutex[worker])     // Lock queue
3. queue.tokens[rear] = 1            // Add token
4. rear = (rear + 1) % MAX_SIZE
5. count++
6. sem_post(queue_mutex[worker])     // Unlock

7. send_fd(unix_sock, client_fd)     // Pass FD
8. sem_post(filled_slots[worker])    // Signal consumer
```

Consumer (Worker) — Dequeue:

```
WORKER: dequeue_token()

1. sem_wait(filled_slots[worker])    // Block if empty

2. sem_wait(queue_mutex[worker])     // Lock queue
3. token = queue.tokens[front]       // Get token
4. front = (front + 1) % MAX_SIZE
5. count--
6. sem_post(queue_mutex[worker])     // Unlock

7. sem_post(empty_slots[worker])     // Signal producer
8. client_fd = recv_fd(unix_sock)    // Receive FD
```

5.3 Thread Pool Synchronization

```
PRODUCER: thread_pool_submit()

pthread_mutex_lock(&mutex)
while (q_count == q_capacity && !shutdown)
    pthread_cond_wait(&cond, &mutex)    // Wait for space

queue[tail] = work_item
tail = (tail + 1) % capacity
q_count++

pthread_cond_broadcast(&cond)           // Wake consumers
pthread_mutex_unlock(&mutex)
```

```
CONSUMER: worker_thread()

while (1):
    pthread_mutex_lock(&mutex)
    while (q_count == 0 && !shutdown)
        pthread_cond_wait(&cond, &mutex)  // Wait for work

    if (shutdown && q_count == 0) break

    item = queue[head]
    head = (head + 1) % capacity
    q_count--

    pthread_cond_broadcast(&cond)         // Wake producer
    pthread_mutex_unlock(&mutex)

    process_client(item.client_fd, ...)   // Outside lock
```

5.4 LRU Cache Synchronization

rwlock Usage:

Operation	Lock Type	Reason
lru_cache_get()	Write lock	Must move entry to front (modify list)
lru_cache_get_copy()	Read lock	Read-only copy, no list modification
lru_cache_put()	Write lock	Insert/update entry, possible eviction

Note: lru_cache_get() uses write lock because accessing an entry moves it to the front of the recency list (LRU update). For concurrent reads without LRU update, use lru_cache_get_copy().

5.5 Statistics Update Synchronization

```
WORKER THREAD: update_stats (after request)

sem_wait(stats_mutex)

stats.total_requests++
stats.bytes_transferred += bytes_sent
stats.status_counts[status_code]++
stats.avg_response_time = rolling_avg(...)

sem_post(stats_mutex)
```

5.6 Lock Ordering (Deadlock Prevention)

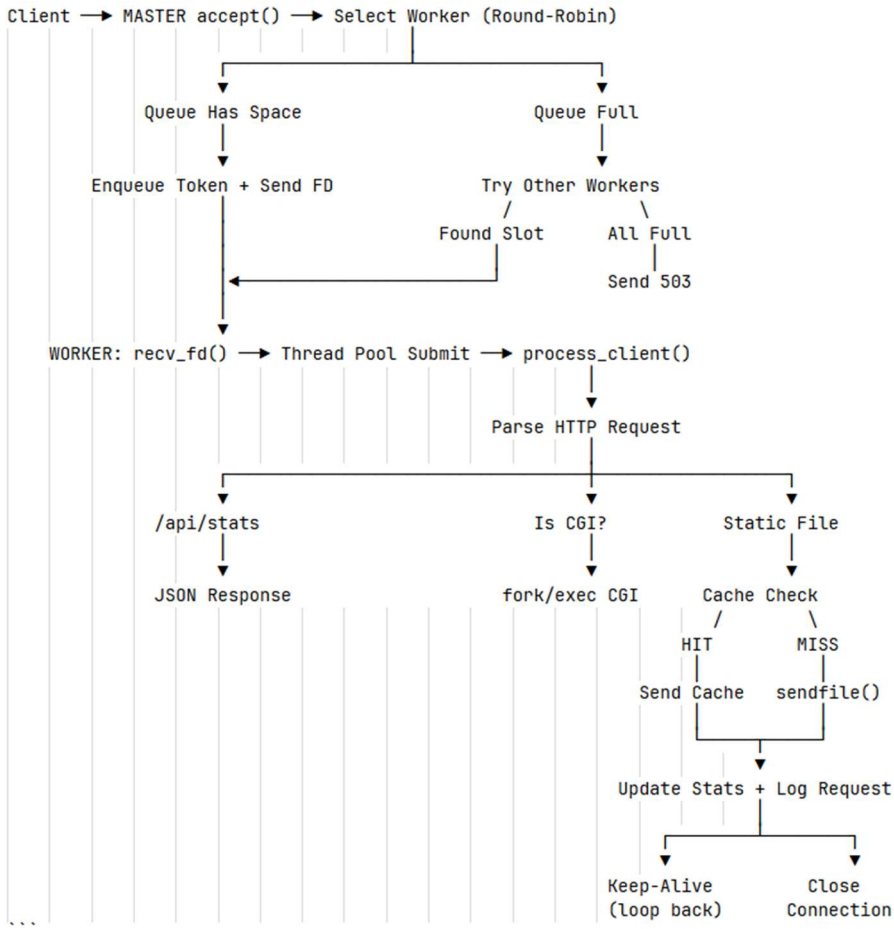
To prevent deadlocks, locks are always acquired in this order:

```
1. stats_mutex      (global statistics)
2. log_mutex        (access log)
3. queue_mutex[i]   (per-worker queue)
4. pool->mutex      (thread pool)
5. cache->lock      (LRU cache)
```

No code path acquires locks in reverse order.

---

6. Request Processing Flow



---

7. Data Structures

7.1 Connection Queue

```
#define  MAX_QUEUE_SIZE  256

typedef struct {
    int  tokens[MAX_QUEUE_SIZE];    // Circular buffer (token = 1)
    int  front;                     // Dequeue index
    int  rear;                      // Enqueue index
    int  count;                     // Current size
}  connection_queue_t;
```

7.2 Server Statistics

```
typedef struct {
    uint64_t  total_requests;       // Total HTTP requests served
    uint64_t bytes_transferred;     // Total bytes sent
    uint64_t cache_hits;            // LRU cache hits
    uint64_t cache_misses;          // LRU cache misses
    uint32_t status_counts[600]; // HTTP status code histogram
    uint32_t active_connections; // Currently open connections
    double avg_response_time;       // Rolling average (ms)
    time_t start_time;              // Server start timestamp
}  server_stats_t;
```

7.3 Thread Pool

```
typedef struct {
    int
    client_fd;
    shared_data_t* data;
    semaphores_t* sems;
}  work_item_t;

typedef struct {
    pthread_t*  threads;            // Thread handles
    int   num_threads;              // Pool size
    pthread_mutex_t   mutex;        // Queue lock
    pthread_cond_t   cond;          // Queue signal
    int   shutdown;                 // Shutdown flag
    work_item_t* queue;             //  Circular  queue
    int   q_head,   q_tail,   q_count;    //   Queue
    pointers int q_capacity;        // Queue size
    void (*worker_func)(...);       // Request handler
}thread_pool_t
```